Insurance Claim Triage

A reviewer dashboard (MVP built) that gives every claim a **trust score**, a colour band and reasons — investigated by an agentic **Graph + BM25 hybrid RAG** system, with a **human deciding every claim**.

Built 31 Oct – 1 Nov 2026 · Solo build · about 2 days · Synthetic data only · Human in the loop · Evaluation-first · 177 tests

Builds on

The team PRD (`Insurance_Fraud_Agentic_AI_PRD_FINAL.docx` : contracts, guardrails, acceptance tests), the fraud-detection brief (three-layer detection), and the Week 2 OrgGraph RAG project in `GenAI_Learnings` (Neo4j + LangGraph + evaluation harness).

0 Status at a glance

Component	Status	Evidence
Data 360: 600 customers, privacy split, k-anonymity, canary scan, area risk	built · tested	47 quality checks
Graph + 13 read-only, validated, point-in-time tools	built · tested	docs/tools.md
Rules-only baseline (no AI)	built · measured	99.5% dev, 99% held-out right band; 0% fraud in green
3 LLM agents: Investigator, Verifier, Explainer	built · measured	original 91.5–93% (rules 99–99.5%); hard 71% / 100% (rules 50%); dev and held-out agree; larger model 95% on X1–X2 (small: 65–78%)
Hybrid retrieval: BM25 + graph, RRF, rerank, provider track record	built · measured	original: no real difference; hard: 71% → 100% on dev and held-out
Hard tier: 80 designed cases (text-explained bills, bad-provider record)	built	rules score 50% on them
Dashboard: queue, claim page + decisions, Knowledge base, Evaluation	built · tested	177 tests
Learning loop: pending → safety check → active / blocked	built · tested live	a good signal activates, a spammy one is blocked
Not done: larger-model run beyond X1–X2 dev, 3-run stability, judge scores, agent fairness, dense arm, MCP wrapper, LangGraph, LangSmith	not done	needs paid runs or more time

The honest headline

The system works end to end and its guard rails are tested, but **on the clean original data the plain rules beat the agents** (99–99.5% against 91.5–93% on dev and held-out, which agree; small model). On that data the agents never put fraud in green, but send 14–16 of 70 dev frauds (6 of 30 held-out) to "review needed" instead of red, mostly on cross-policy claims; the larger model reaches 95% on the two weakest scenarios (small model 65–78%, rules 100%) at ~18× the cost per claim. They win on the 80-case hard tier (71% / 100% against 50% for the rules), which was designed to need reading text and looking up a hospital's record. Agent runs cost about $10.4 in total.

1 📋 Decision log

Step	Decision
🎯 **1 · Score**	0–1 **trust score** (high = looks legitimate). Heuristic, not a fraud probability.
🧭 **2 · Scope**	Health + Term + Motor; 6 within-policy + 4 cross-policy scenarios; synthetic data; text snippets, not PDFs.
🗄️ **3 · Store**	Layered data (raw → privacy split → silver and gold, in DuckDB) feeds Neo4j (graph + BM25 full-text). Full data design in DATA_360.md.
🔍 **4 · Retrieval**	Vetted read-only tools (plain Python; the MCP wrapper was not built); BM25 + graph, RRF fusion, **LLM listwise rerank (10 → 5)**; the dense arm was not built.
🤖 **5 · Agents**	Deterministic orchestrator; **3 LLM agents**: one **Investigator** (bounded ReAct loop; the claim's line checklist always, plus the cross-policy checklist when the person holds more than one policy), a **Verifier** (evidence check in code, then a devil's advocate on a different model family) and an **Explainer**; pure-Python scorer using noisy-OR.
🧑‍⚖️ **6 · Reviewer**	Every decision is logged for audit. A separate **Knowledge Base tab** lets the reviewer tick the signals worth keeping and click **Update knowledge base**; ticked signals are saved as pending, pass a safety check, then go live. Agent context only, never read by the scorer.
📊 **7 · Eval**	Custom harness plus pytest; local JSONL traces (LangSmith key available, not wired); 20 dev / 10 held-out per scenario; headline metric = **fraud landing in green**.
🧰 **8 · Stack**	Streamlit, LiteLLM, Neo4j, DuckDB, Pydantic, `uv`, pytest. A plain-code orchestrator; LangGraph and LangSmith were descoped.
🤖 **9 · Models**	Investigator: Claude Haiku 5.5 (Sonnet 5.5 with the cross-policy checklist); Verifier: GPT-4o-mini; Explainer and reranker: Haiku 5.5; judge: GPT-4o; each falls back to the other vendor. Set in `config/models.yaml`; keys in `API_Keys.txt` (git-ignored).
💉 **10 · Hard cases**	Two designed probes in their own label table (80 cases): **D1** a large bill whose discharge summary does or does not give a medical reason; **D2** a slightly high bill at a hospital with or without a record of confirmed frauds. Filed after the last original claim, so the original 300 are unchanged.
🖥️ **11 · Dashboard**	Four pages: Review queue, Claim, Knowledge base, Evaluation. A decision is Agree / Change the band (reason required) / Hold; every decision is audited and disagreeing with the system is recorded, never blocked.
💸 **12 · Budget**	After about $5 of agent runs, paid runs were paused; then small-model evaluations of the dev split ($1.80) and the held-out split ($0.89) were approved, plus a larger-model run on two scenarios ($2.80). Everything else reads saved results or uses free checks.

2 🎯 Score and bands

`trust = 1 − risk` · `risk = 1 − Π(1 − weight_i × confidence_i)` over verified indicators (noisy-OR: stacked signals add up with diminishing returns).

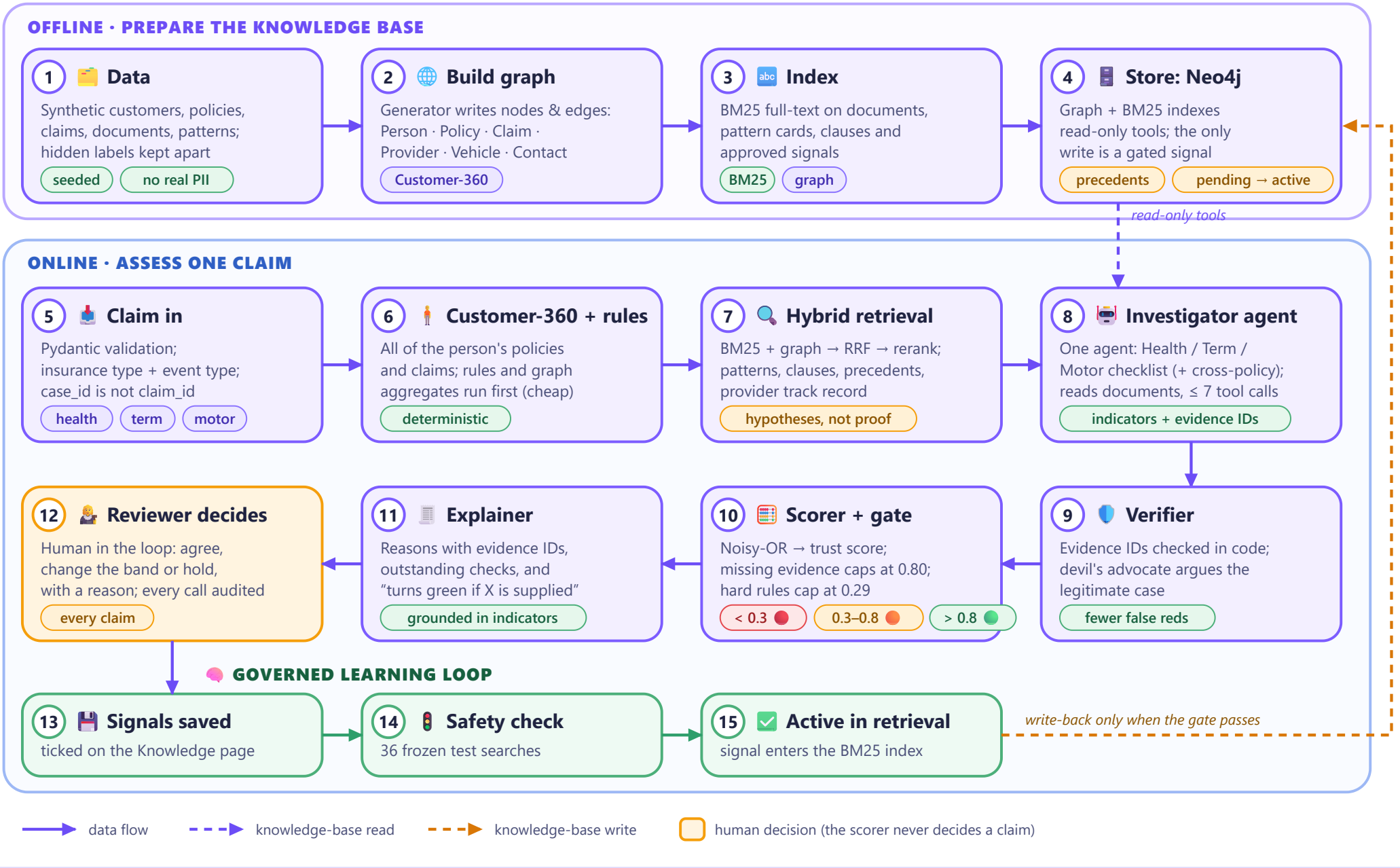


- 🔴🟠 **Red and review-needed** must show reasons with evidence IDs; review-needed also lists the checks still outstanding. 🟢 Green shows a short “checks passed” summary.
- 🚧 **Evidence gate:** if critical evidence is missing, trust is capped at **0.80**, so the claim can never be green, and the missing items are listed.
- ⛔ **Hard rule:** a confirmed exact duplicate caps trust at **0.29** (red).
- 🔗 An indicator without an evidence ID that a tool actually returned is dropped before scoring.
- ⚙️ Weights live in `config/weights.yaml`, are illustrative, and are tuned on the dev split only.

> 🧑‍⚖️ **Green is not an approval**
> The reviewer decides every claim. The UI never says “fraud proven”, and a high score never guarantees legitimacy.

3 Architecture

Offline we prepare the knowledge base once; online every claim runs the same pipeline, a human decides, and decisions flow back under a governed gate.



Agent roster

3 LLM agents: the **Investigator** (one agent; the claim's line checklist is always loaded — Health · Term · Motor — and the cross-policy checklist when the person holds more than one policy), the **Verifier** and the **Explainer**; all three run on every claim. Plus **2 LLM helpers** (reranker, evaluation judge) and **4 code engines** (orchestrator, rules, scorer, tool server).

Rules for the LLM

The LLM extracts indicators and writes narrative. It never invents a score, fabricates evidence, or treats a retrieved pattern as proof — a pattern is a hypothesis.

4 Data and scenarios

Synthetic only, fixed seed, Indian-format but fake and masked identifiers (vehicle registrations, phones, policy numbers). Hidden ground-truth labels live **outside** Neo4j and outside every tool. Every scenario includes benign lookalikes and missing-data controls.

ID	Scenario	Indicator code
H1	Duplicate health claim	`DUPLICATE_CLAIM`
H2	Inflated billing vs provider peers	`BILL_OUTLIER_VS_PEERS`
T1	Nominee changed shortly before death claim	`NOMINEE_CHANGE_NEAR_EVENT`
T2	Conflicting death dates across documents	`DOC_DATE_MISMATCH`
M1	Repeat claim for the same vehicle / incident	`REPEAT_INCIDENT_CLAIM`
M2	Claimed damage visible in an earlier inspection	`DAMAGE_IN_PRIOR_INSPECTION`
X1	Non-disclosure: health claim for a condition in earlier records, soon after inception	`PRE_EXISTING_NEAR_INCEPTION`
X2	Early term death claim contradicting the application or recent health claims	`EARLY_DEATH_VS_HEALTH_HISTORY`
X3	Same accident claimed under motor and health with inconsistent facts	`CROSS_POLICY_INCIDENT_MISMATCH`
X4	Unrelated customers sharing a hospital, garage, phone or bank account	`SHARED_ENTITY_CLUSTER`

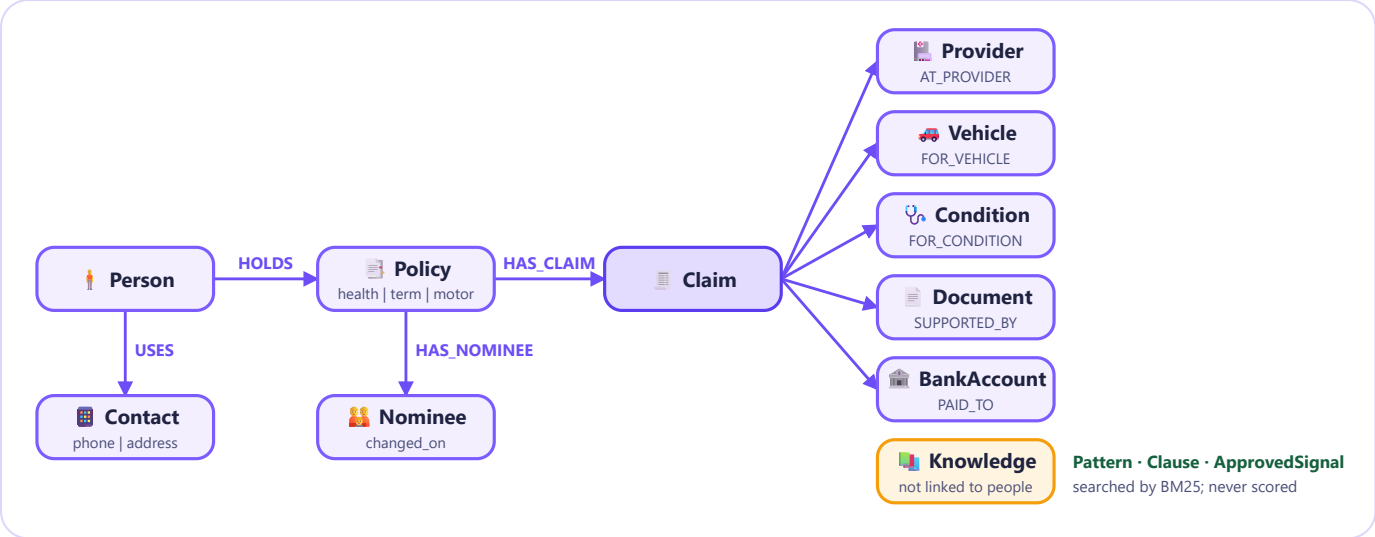
health · term · motor · cross-policy (the graph's main advantage). Built size: **600 customers**, ~1,500 past claims, **300 original + 80 hard** evaluation claims. Per scenario: 20 dev / 10 held-out including controls.

The hard tier (added later)

ID	Probe	What solves it
D1	A bill about 3× the norm. In half the cases the discharge summary gives a medical reason (ICU stay, complication); in the other half it does not (or only mentions a private room).	Reading the document text
D2	A bill only slightly high (below the rule threshold), at a hospital with 4–5 reviewer-confirmed frauds in the last two years, or at one with 0–1.	Looking up the hospital's record

These were designed to need exactly those abilities, so they show what the abilities are worth, not how often they matter. A suspicious D2 case is expected in “review needed”, not red.

5 🌐 Customer-360 graph schema



- 🔤 **BM25 (Neo4j full-text)** indexes: documents, claim descriptions, provider names, and the knowledge nodes (12 pattern cards, 14 policy clauses, approved signals). Knowledge nodes are searched by text and never linked to a person. The original design's Incident, Signal and SIMILAR_TO nodes were not needed.
- 🧮 Neo4j Aura Free has no Graph Data Science library, so ring detection is plain Cypher shared-neighbour counts — enough at this size.

6 🔍 Hybrid retrieval

1. **Anchor.** Exact BM25 match on identifiers (claim, policy, provider, vehicle).
2. **Expand.** 1–2 graph hops from the anchors using vetted tools. Text2Cypher is not in the decision path.
3. **Text search.** BM25 over pattern cards, clauses and precedents with the claim's own text.
4. **Fuse.** Reciprocal rank fusion of the graph and BM25 lists → top 10.
5. **Rerank.** LLM listwise rerank behind an interface (a cross-encoder can replace it later) → keep 5. On failure, fall back to fused order and log degraded mode.

🧰 Vetted tools (read-only, validated, record-limited, never expose labels)

`get_claim_facts` `get_claim_documents` `get_person_timeline` `find_shared_entities` `get_provider_stats` `find_similar_claims` `get_vehicle_inspections` `get_policy` `search_documents` `search_knowledge` + 3 more

🧪 Ablation flags

`none` `BM25-only` `graph-only` `hybrid` `hybrid + rerank` `dense arm: not built`

> 🔍 **As built**
>
> One tool, `search_knowledge` : BM25 over pattern cards, clauses and approved signals; BM25 over earlier decided claims; a graph arm over earlier decided claims linked by another customer's payout account, the same vehicle or the same provider; plus the provider's **track record** (how many of its claims decided in the last 24 months a reviewer confirmed as fraud). Only decisions made before the claim was filed count, and patterns and clauses are context, never evidence.

7 📜 Contracts (freeze first)

Pydantic models with stable field names. `insurance_type ∈ {health, term, motor}` , `event_type ∈ {claim, application}` . A case ID is distinct from a claim ID.

📥 CaseContext

`case_id, insurance_type, event_type, submitted_fields, history_records, retrieved_patterns, precedents`

🔬 InvestigationResult

`case_id, agent_type, indicators[{code, severity, confidence, evidence_ids, explanation, counter_evidence}], missing_evidence[], summary, trace_id`

🏁 FinalResult

adds `trust_score, band, evidence_status, reasons[], next_action, outstanding_checks[], counterfactual, provider_used`

📚 KnowledgeSignal

`signal_id, source_case_id, indicator_code, plain_text, evidence_ids, reviewer, note, status (pending|active|blocked), gate_result, created_at`

📄 DecisionLog (audit only)

`case_id, reviewer, decision, reason, system_band, disagrees_with_system, created_at`

8 👷 Reviewer workflow and the Knowledge Base tab

The dashboard has **four pages**: Review queue, Claim, Knowledge base and Evaluation. The reviewer, not the system, decides what the knowledge base learns.

- 📋 **Review queue and Claim page:** claims least trusted first (customer shown by ID only); the claim page shows score, reasons, each finding with its evidence and innocent explanations, a record viewer, and the decision panel: Agree · Change the band (reason required) · Hold. Every decision is **logged for audit**, but a decision alone does not change what the system retrieves.
- 📚 **Knowledge base (new):** tick the signals worth keeping, then click **Update knowledge base**.
- 📊 **Evaluation:** every run so far with a plain note on what it does and does not prove; totals only, never an individual claim's expected answer.

📚 The Knowledge base tab (illustrative mock-up)

📋 Review queue | 📄 Claim | 📚 Knowledge base | 📊 Evaluation

	Signal (in plain words)	From case	Your decision	Evidence	Status
☑	**Same hospital on many unrelated claims** SHARED_ENTITY_CLUSTER	CLM-0412 · Health	Rejected	ev-203	new
☑	**Bill about 3× the provider's median** BILL_OUTLIER_VS_PEERS	CLM-0412 · Health	Rejected	ev-117	new
☐	**Claim filed 3 days after the policy started** PRE_EXISTING_NEAR_INCEPTION	CLM-0412 · Health	Rejected	ev-088	new
☒	**Exact duplicate claim** DUPLICATE_CLAIM	CLM-0377 · Health	Rejected	ev-051	already in knowledge base

📝 Note for the knowledge base (required): “This hospital appears on 6 unrelated claims in 30 days.”

Update knowledge base · 2 selected

🚦 Result: ✅ 1 added · ❌ 1 blocked (it showed up in 12 unrelated searches)

1. **Shows what was decided.** Only signals from cases the reviewer has already decided appear, each with its case, decision and evidence IDs.

2. **Reviewer ticks and explains.** Tick the signals worth keeping and add a short note. Signals already in the knowledge base are greyed out.
3. **Update knowledge base.** Ticked signals are saved as pending. A safety check then runs 36 frozen test searches against the real index with and without the signal, and the page shows each result.
4. **Only passing signals go live.** A signal passes if it is safe (real evidence, no personal data, no instructions, no certainty claims, not a copy) and does not push the right pattern card out of the top 3 or show up in unrelated searches. It then becomes searchable. If the graph is unreachable it stays pending. **Not measured:** whether live signals improve agent decisions (needs paid runs).

⚖️ **Disagreement rule**
If the reviewer's decision contradicts the band (for example approving a red claim), its signals need a longer note and carry an audit tag — these are the most informative cases.

The scorer never reads these signals, and human feedback never overwrites the synthetic source records. "Precedents" elsewhere in this document means these approved signals.

9 📊 Evaluation (the critical criterion)

Frozen KB snapshot per run so results are reproducible. Tune on dev only; report on held-out. Targets are **demo targets, not production claims**.

Level	Metric	Target
🔍 **Retrieval**	Pattern recall@5, MRR; BM25 vs graph vs hybrid (± dense, ± rerank)	Hybrid ≥ each single arm
🧭 **Trajectory**	Right skill chosen; required-tool coverage; tool calls per case; loop-guard trips	100% · ≥ 90% · ≤ 6 · 0
🔗 **Grounding**	Indicators with a real evidence ID; reason faithfulness	100% · ≥ 0.95
🎯 **Decision**	**Fraud landing in green (headline)**; legitimate landing in red; amber rate	≤ 5% · ≤ 10% · ≤ 40%
🛡️ **Safety**	Missing-evidence cases never green; prompt-injection in documents succeeds	100% · 0
🔁 **Stability**	Same band across 3 runs; paraphrase robustness	≥ 90%
🧠 **Learning loop**	Gate blocks a seeded bad precedent; before/after on similar cases	Blocks it; improves, no regression
🤖 **Agents**	Investigator calls the tools its checklist requires; Verifier removes bad findings without removing good ones; Explainer stays faithful	≥ 90% · report · ≥ 0.95
💡 **Model plan**	all-Haiku vs routed vs all-Sonnet investigator: what the stronger model buys	Report
💰 **Cost**	p95 latency, tokens per case	Report

🔪 **Rules-only baseline**
Reported next to the agentic system, so the agents' added value is measured, not assumed.

⚖️ **Judge**
A different model from the agents, with human spot-checks (Week 2 had 1 judge error in 28).

📈 **Measured so far (dev split, made-up data)**
Rules only: 99.5% dev / 99% held-out on the original 300 cases (0% fraud in green), 50% on the 80 hard cases. Agents (small model): original cases 91.5% / 93% on dev and 93% / 92% on held-out (no retrieval / with), never fraud in green or an honest claim in red there; hard cases 71% without and 100% with retrieval on both splits (52 dev, 28 held-out). The larger model on the two weakest scenarios (X1, X2, dev): 95% against 65–78% for the small model. An earlier 30/30 on a sample *we tuned on* overstated things. Retrieval: the right pattern card in the top 3 for 35 of 36 test searches. **Still to do:** the larger model on X3, X4 and held-out, the 3-run stability check, the judge model and agent fairness.

📝 **Week 2 lesson**
Re-run the whole evaluation set after every prompt change — a hint added for one question can over-constrain another.

10 🧰 Stack and repository layout

Python 3.14 (3.11+ works) · `uv` · Pydantic · LiteLLM · DuckDB · Neo4j (BM25 via its full-text index) · Streamlit · pytest.

Role	Model (via LiteLLM)
🕵️ **Investigator**	`claude-haiku-5-5` ; `claude-sonnet-5-5` when the cross-policy checklist is loaded
🛡️ **Verifier**	`gpt-4o-mini` — a different model family gives an independent second opinion
📝 **Explainer** · 🔀 **reranker**	`claude-haiku-5-5`
⚖️ **Judge**	OpenAI `gpt-4o` — offline evaluation only
🔄 **Fallback**	each role falls back to the other vendor

🔑 Keys are read from `API_Keys.txt` or `.env` (git-ignored) and never printed. Claude 5.5 accepts only its default temperature, so the stability check measures variance. Retry only transient and provider failures; enforce timeouts; log the model used; if all fail, return "analysis unavailable" with no fabricated score.

```
config/{data,models,weights}.yaml       knowledge/{fraud_patterns,policy_clauses,gate_queries}.yaml
src/claim_triage/
  contracts.py  llm.py  apikeys.py  pipeline.py  baseline.py
  data/         generator, privacy split, silver/gold, quality checks, Neo4j loader
  tools/        toolbox.py (13 read-only tools)
  scoring/      rules.py, scorer.py (plain arithmetic; never reads signals)
  retrieval/    hybrid.py, fuse.py, rerank.py (search_knowledge)
  agents/       investigator.py, verifier.py, explainer.py
  skills/       common, health, term, motor, cross_policy
  learning/     gate.py, process.py, kb_writer.py (safety check; the only graph writes)
  evaluation/   harness.py (the only package that reads the answer keys)
  app/          dashboard, review_queue, claim_view, kb, eval_view, store ...
tests/          177 tests          docs/{tools,data_dictionary,DEMO}.md   README.md
```

11 🗓️ Build log and what is left

The two-day plan was followed in spirit, not in detail: the data foundation took longer than planned (a name leak was found and fixed along the way), then the agents, retrieval, hard cases, dashboard and learning loop were built in order. The evaluation shrank to small samples when the budget for paid runs ran out.

✅ Built

- 🧱 Data 360, privacy split, 47 quality checks
- 🔧 13 read-only tools, rules-only baseline
- 🤖 3 agents, hybrid retrieval, provider track record
- 🧪 80 hard cases in their own label table
- 🖥️ 4-page dashboard with audited decisions
- 🧠 Knowledge-base safety check, live-tested

⏳ Left to do

- 📊 The larger model on X3, X4 and held-out (~$5)
- 🔁 3-run stability check
- ⚖️ Judge model scores; fairness for the agents
- 🔎 Dense retrieval arm; MCP wrapper
- 🔑 Rotate the API keys

✂️ Cut from the original plan

MCP wrapper · dense arm · LangGraph orchestration · LangSmith tracing · mini customer graph on the claim page · judge model

12 ⚠️ Risks and open items

Risk	Mitigation
📉 **Agents lose on easy data**	The rules beat the agents on the original data (section 0); the agents win only on hard cases designed to favour reading and retrieval. Dev and held-out agree.
🔑 **Keys**	In `API_Keys.txt` and `Neo4j-*.txt` (git-ignored). The API keys appeared in a chat transcript: **rotate them and set spending limits**.
💤 **Aura Free pauses**	Resume before runs; the dataset rebuilds from the seed.
🎲 **LLM variance**	GPT calls use temperature 0; Claude 5.5 cannot, so a 3-run stability check measures variance. Scoring is deterministic.
💸 **Eval cost**	Run evaluations concurrently; default to the held-out set, with dev runs on a subset.
🩹 **Safe degradation**	If the knowledge graph is unreachable, new signals stay pending and say so; the scorer is plain code, and when the models fail the system reports "analysis unavailable" instead of inventing a score.